

Тема 1. Переменные, типы данных, основы синтаксиса JavaScript

Инструкция (statement)

Инструкция (statement) — законченная команда для машины. Заканчивается точкой с запятой `;`.

```
a = b * 2;
```

- `a`, `b` — переменные (variables): хранилища данных (имя + значение);
- `2` — литерал (literal): значение, записанное прямо в коде;
- `=`, `*` — операторы (operators): выполняют действия над значениями.

Как выполняется: находим значение `b` → подставляем вместо `b` → умножаем на `2` → результат записываем в `a`.

Выражение (expression) и литерал (literal)

Выражение (expression) — ссылка на переменную/значение или их комбинация с оператором. В инструкции `a = b * 2;` пять выражений: `2` (литерал), `b` и `a` (переменные), `b * 2` (арифметическое выражение), `a = b * 2` (выражение присваивания).

Литерал (literal) — значение, заданное прямо в коде: числовой литерал (число) или строковый литерал (текст в кавычках).

Подключение скрипта к HTML

Встроенный скрипт

Код JavaScript пишется прямо внутри тега `<script>`, обычно в `<head>`.

```
<script>
  console.log("Hello, world");
</script>
```

Внешний скрипт

Код выносится в отдельный файл `.js` и подключается через атрибут `src`:

```
<script src="my-script.js" defer></script>
```

Атрибут `defer` откладывает выполнение скрипта до полной загрузки HTML-документа — страница не тормозит при отображении.

Внешние скрипты предпочтительнее встроенных — код получается более читабельным, удобным для поддержки и переиспользования.

Строгий режим (strict mode)

Специальный режим выполнения, приводящий код к современному стандарту и запрещающий небезопасные устаревшие конструкции. Включается директивой в начале скрипта:

```
'use strict';
```

Рекомендуется использовать всегда.

Вывод данных: console.log()

Для проверки работы кода используется консоль браузера (вкладка Console, открывается Ctrl+Shift+J на Windows/Linux или Cmd+Option+J на Mac).

```
console.log("JavaScript is awesome!");
console.log(10);
```

Переменные и типы данных

Объявление переменных

Переменная (variable) — контейнер для данных: идентификатор (identifier) (имя) + область памяти со значением.

```
const age = 20;
const username = "Mango";
```

- Объявление начинается с ключевого слова (keyword): `const` или `let` ;
- оператор `=` присваивает значение;
- инструкция заканчивается `;` .

Переопределение значения

Значение можно изменить только у переменной, объявленной через `let` :

```
let username = "Mango";
username = "Poly";
```

`const` переопределить нельзя — будет ошибка:

```
const username = "Mango";
username = "Poly"; // TypeError: Assignment to constant variable.
```

Если переменной `let` не задать значение сразу, она получает значение `undefined` (не определено).

Как выбрать между `const` и `let`

По умолчанию используй `const` — код получается надёжнее и понятнее, так как значение не меняется. Используй `let` , если значение переменной должно изменяться в процессе выполнения (счётчики, временные значения).

Именование переменных

1. Разрешены буквы (a-z, A-Z), цифры (0-9), символ подчёркивания `_` и знак доллара `$` .
2. Первый символ — буква, `_` или `$` (не цифра).
3. Регистр важен: `user` , `usEr` , `User` — разные переменные.
4. Имя должно быть понятным и описывать назначение (не `chislo` , а `number`).
5. Стандарт именования — camelCase: `userProfile` , `isActive` , `getUserData` .

Нельзя использовать зарезервированные ключевые слова языка в качестве имён переменных.

Типичные ошибки, типы данных и арифметика

Типичные ошибки (Errors)

Ошибки при выполнении кода отображаются в консоли разработчика и содержат описание — где и почему возникла проблема. Работа с консолью и чтение ошибок — базовый навык отладки (debugging).

Неправильное имя переменной

Обращение к необъявленной переменной:

```
const username = "Mango";
console.log(user); // ReferenceError: user is not defined
```

Обращение к переменной до её объявления

Переменные, объявленные через `const` / `let`, недоступны до момента объявления:

```
console.log(age); // ReferenceError: age is not defined
let age = 20;
```

Переопределение const

Попытка присвоить новое значение переменной `const` вызывает ошибку `TypeError` (см. раздел «Переопределение значения» выше).

Типы данных (Data Types)

Примитивные типы данных (primitive data types) — базовые виды информации, с которыми работает программа: числа, текст, логические значения и т.д. Переменная в JavaScript не привязана к одному типу — может хранить значения разных типов.

Number (число)

Целые и дробные числа. Целая и дробная часть разделяются точкой (не запятой — иначе ошибка):

```
const age = 20;
const salary = 3710.84;
```

String (строка)

Последовательность символов в одинарных `' '` или двойных `" "` кавычках:

```
const username = 'Mango995';
const description = "JavaScript is awesome!";
```

Boolean (логический тип)

Только два значения: `true` и `false`, без кавычек. `true` — логическое значение, `"true"` — строка. Используется для да/нет-условий:

```
const isModalOpen = true;
const isLoggedIn = false;
```

Переменные булевого типа обычно называют как вопрос («да/нет»): `isLoggedIn`, `isModalOpen`.

Специальные значения: null и undefined

Оба означают отсутствие значения, но по-разному:

- `null` — явно означает «пусто», разработчик сам присваивает это значение, когда точно известно, что данных нет;
- `undefined` — присваивается автоматически: если переменная объявлена, но не инициализирована, либо явно указано `undefined`. Означает «значение пока неизвестно/не задано».

```
let value = null;
console.log(value); // null

let value2;
console.log(value2); // undefined
```

Оператор typeof

Определяет тип значения/выражения, возвращает строку с названием типа:

```
console.log(typeof 17);           // "number"
console.log(typeof "text");       // "string"
console.log(typeof false);        // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object"
```

Важный нюанс для собеседований: `typeof null` возвращает `"object"`, хотя `null` — примитивное значение, а не объект. Это историческая ошибка в реализации языка, сохранённая ради обратной совместимости.

Арифметические операции

Действуют обычные правила порядка операций (сначала скобки, потом степени, потом умножение/деление и т.д.).

Оператор	Действие	Пример	Результат
+	сложение	8 + 5	13
-	вычитание	8 - 5	3
*	умножение	8 * 5	40
/	деление	8 / 5	1.6
%	остаток от деления	8 % 5	3
**	возведение в степень	8 ** 5	32768

Комбинированные операторы (Compound assignment operators)

Позволяют компактно обновлять значение переменной на основе её текущего значения:

Оператор	Пример	Эквивалент
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Пример:

```
let age = 25;
age += 1;
console.log(age); // 26
```

Строки

Конкатенация строк (concatenation)

Если применить оператор `+` к строке и любому другому типу данных, результатом будет новая строка — объединение исходных значений. Это называется конкатенация (concatenation, склеивание).

```
const message = "Mango " + "is" + " happy";
console.log(message); // "Mango is happy"
```

В конкатенацию можно подставлять значения переменных:

```
const age = 24;
const message = "Poly is " + age + " years old!";
```

Любой тип данных при конкатенации приводится к строке:

```
console.log("Mango" + 55); // "Mango55"
console.log("Mango" + true); // "Mangotrue"
```

Важен порядок операндов — преобразование в строку происходит только в момент сложения со строкой, до этого работает обычная математика:

```
console.log(1 + "2"); // "12"
console.log(1 + "2" + 4); // "124"
console.log(1 + 2 + "4"); // "34"
```

Преобразование типов (type conversion)

Процесс изменения значения одного типа данных в другой. Бывает двух видов: явное (explicit) и неявное (implicit).

Явное преобразование (explicit conversion)

Выполняется программистом осознанно. Функция `String()` приводит значение к строке:

```
console.log(String(5)); // "5"
console.log(String(true)); // "true"
console.log(String(null)); // "null"
console.log(String(undefined)); // "undefined"
```

Неявное преобразование (implicit conversion)

Происходит автоматически. Например, при `+` со строкой JS сам приводит второй операнд к строке:

```
console.log("5" + 3); // "53"
console.log("5" + true); // "5true"
console.log("5" + null); // "5null"
console.log("5" + undefined); // "5undefined"
```

Неявное преобразование удобно, но может давать неожиданные результаты — важно быть внимательным при операциях с разными типами данных.

Шаблонные строки (template strings / template literals)

Синтаксис для удобного объединения статичного текста с динамическим (переменными, вычислениями), без громоздкой конкатенации. Оборачиваются обратными кавычками — backticks (```), а не обычными `'` или `"`.

Значения переменных подставляются через интерполяцию (interpolation) — конструкцию `${переменная}`:

```
const guestName = "Mango";
const roomNumber = 207;
const greeting = `Welcome ${guestName}, your room number is ${roomNumber}!`;
console.log(greeting); // "Welcome Mango, your room number is 207!"
```

Сравнение с конкатенацией — тот же результат, но менее читабельно:

```
const greeting = "Welcome " + guestName + ", your room number is " + roomNumber + "!";
```

Чтобы напечатать backtick — переключись на английскую раскладку клавиатуры, клавиша ``` / `~` обычно слева от «1».

Длина строки: свойство length

Свойство (property) — описательная характеристика сущности (entity). Для доступа к свойству используется точка: `сущность.свойство`.

Строка имеет встроенное свойство `length` — возвращает количество символов в строке:

```
const productName = "Repair droid";
console.log(productName.length); // 12
console.log("Repair droid".length); // 12
```

Индексация строк (string indexing)

Индексация (indexing) — нумерация символов строки, начинается с 0. Первый символ — индекс `0`, последний — `length - 1`.

```
const product = "Repair droid";
console.log(product[0]); // 'R'
console.log(product[5]); // 'r'
console.log(product[11]); // 'd'
```

Доступ к последнему символу — через `length - 1`:

```
const product = "Repair droid";
console.log(product[product.length - 1]); // 'd'
```

Неизменность строк (string immutability)

Строка после создания неизменна (immutability) — нельзя заменить отдельный символ. Присвоение по индексу не даёт эффекта:

```
let product = "Droid";
product[2] = "0";
console.log(product); // "Droid" (без изменений)
```

Чтобы изменить строку, нужно присвоить переменной новое значение целиком:

```
let product = "Droid";
product = "Dr0id";
console.log(product); // "Dr0id"
```

Операторы сравнения и преобразование в числа

Операторы сравнения (comparison operators)

Сравнивают два значения, возвращают `true / false`: `>` (больше), `<` (меньше), `>=` (больше или равно), `<=` (меньше или равно).

```
const a = 2;
const b = 5;
console.log(a > b); // false
console.log(a < b); // true
console.log(a >= b); // false
console.log(a <= b); // true
```

Операторы равенства (equality operators)

Нестрогое равенство (loose equality): `==` и `!=` — перед сравнением может приводить операнды к одному типу, что часто даёт неожиданный результат:

```
console.log(5 == "5"); // true (строка приводится к числу)
console.log(1 == true); // true (true приводится к 1)
```

Строгое равенство (strict equality): `===` и `!==` — сравнивает и значение, и тип, без приведения. Рекомендуется использовать именно их:

```
console.log(5 === "5"); // false — разные типы
console.log(1 === true); // false — разные типы
console.log(5 === 5); // true
```

Преобразование типов: числа

Функция `Number()` — явное преобразование в число:

```
console.log(Number("5")); // 5
console.log(Number(true)); // 1
console.log(Number(false)); // 0
console.log(Number(null)); // 0
```

`true` → `1`; `false`, `null`, `""` → `0`. Если преобразовать невозможно — результат `NaN` (Not a Number, «не число»):

```
console.log(Number(undefined)); // NaN
console.log(Number("Jacob")); // NaN
console.log(Number("25px")); // NaN
```

Неявное преобразование в арифметике: если хотя бы один операнд — строка, оба приводятся к числу (кроме `+`, где строка вызывает конкатенацию):

```
console.log("5" * 2); // 10
console.log("10" - 5); // 5
console.log(5 + true); // 6
```

В операторах сравнения (`<`, `>`, `<=`, `>=`) тоже происходит неявное приведение к числу:

```
console.log("10" > 5); // true
console.log(5 > true); // true
```

Преобразование строк в числа: `parseInt` / `parseFloat`

`Number.parseInt(строка, система_счисления)` — разбирает строку слева направо, забирает цифры до первого недопустимого символа, возвращает целое число:

```
console.log(Number.parseInt("5")); // 5
console.log(Number.parseInt("5.5")); // 5
console.log(Number.parseInt("12qwe74")); // 12
console.log(Number.parseInt("qweqwe")); // NaN
```

`Number.parseFloat(строка)` — то же самое, но сохраняет дробную часть:

```
console.log(Number.parseFloat("3.14")); // 3.14
console.log(Number.parseFloat("5.5cm")); // 5.5
console.log(Number.parseFloat("qweqwe")); // NaN
```

Математические функции (объект `Math`)

Встроенный объект `Math` с методами для работы с числами:

Метод	Что делает	Пример
<code>Math.floor(num)</code>	округление вниз	<code>Math.floor(1.7)</code> → 1
<code>Math.ceil(num)</code>	округление вверх	<code>Math.ceil(1.3)</code> → 2
<code>Math.round(num)</code>	округление к ближайшему	<code>Math.round(1.7)</code> → 2
<code>Math.max(...)</code>	максимум из набора	<code>Math.max(20, 10, 50)</code> → 50
<code>Math.min(...)</code>	минимум из набора	<code>Math.min(20, 10, 50)</code> → 10
<code>Math.random()</code>	случайное число от 0 до 1 (не включая 1)	напр. 0.2...0.916...

Дробные числа: погрешность вычислений

Из-за особенностей хранения чисел в двоичной системе дробные вычисления могут давать неточность:

```
console.log(0.1 + 0.2 === 0.3); // false
console.log(0.1 + 0.2);          // 0.30000000000000004
```

Способ 1 — умножить на 10/100, сложить, разделить обратно:

```
console.log((0.1 * 10 + 0.2 * 10) / 10); // 0.3
```

Способ 2 — метод `toFixed(n)`, округляет до `n` знаков после точки, возвращает строку:

```
console.log((0.1 + 0.2).toFixed(1)); // "0.3"
console.log((8.762195).toFixed(4));  // "8.7622"
```

Основы функций

Объявление и вызов функции (function declaration & function call)

Функция — независимый блок кода, который выполняет задачу с разными входными данными. Можно представить как чёрный ящик: принимает данные на входе, возвращает результат на выходе.

Объявление функции (function declaration):

1. ключевое слово `function`;
2. имя функции — глагол, отвечающий на вопрос «что сделать?»;
3. круглые скобки `()`;
4. тело функции (function body) в фигурных скобках `{ }`.

```
function doStuff() {
  console.log('Log inside multiply function');
}
```

Вызов функции (function call): выполняется по имени с круглыми скобками:

```
doStuff(); // 'Log inside multiply function'
doStuff(); // можно вызывать сколько угодно раз
```

Параметры и аргументы (parameters & arguments)

Параметры (parameters) — переменные в скобках при объявлении функции, локальны (доступны только внутри тела функции):


```
function multiply(x, y, z) {
  console.log(`Result: ${x * y * z}`);
}
```

Аргументы (arguments) — значения, которые передаются при вызове функции вместо параметров. Порядок важен: первый аргумент → первому параметру, второй → второму и т.д.

```
multiply(2, 3, 5);    // "Result: 30"
multiply(4, 8, 12);   // "Result: 384"
```

При каждом вызове параметры создаются заново — значения из разных вызовов друг на друга не влияют.

Возврат значения: return

Оператор `return` возвращает значение из функции в место её вызова и немедленно прекращает выполнение функции.

```
function multiply(x, y, z) {
  const product = x * y * z;
  return product;
}
const result = multiply(2, 3, 5);
console.log(result); // 30
```

Результат выражения можно возвращать сразу, без промежуточной переменной:

```
function multiply(x, y, z) {
  return x * y * z;
}
```

Если `return` отсутствует или не указывает значение — функция вернёт `undefined`:

```
function multiply(x, y, z) {
  const product = x * y * z; // return не указан
}
console.log(multiply(2, 3, 5)); // undefined
```

Код после `return` не выполняется — выполнение функции обрывается сразу на `return`:

```
function multiply(x, y, z) {
  console.log('Этот код выполнится');
  return x * y * z;
  console.log('А этот — никогда, он после return');
}
```

Порядок выполнения кода

При вызове функции интерпретатор останавливает текущий код, выполняет тело функции, затем возвращается в точку вызова и продолжает выполнение оставшегося кода.

```
function multiply(x, y, z) {
  console.log(`Result: ${x * y * z}`);
}

console.log("Log before multiply execution");
multiply(2, 3, 5);
console.log("Log after multiply execution");
```

Порядок логов: "Log before multiply execution" → "Result: 30" → "Log after multiply execution".

Область видимости функции (scope)

Область видимости (scope) определяет, где в коде доступны переменные и функции.

Глобальная область видимости (global scope): переменные, объявленные вне любых блоков кода, доступны в любом месте программы — это глобальные переменные (global variables).

```
const value = "I'm a global variable";

function foo() {
  console.log(value); // "I'm a global variable" — виден снаружи
}

foo();
console.log(value); // "I'm a global variable"
```

Локальная область видимости (local scope): любая конструкция с фигурными скобками `{ }` (условия, циклы, функции) создаёт свою локальную область. Переменные, объявленные внутри, доступны только там.

```
function foo() {
  const value = "I'm a local variable";
  console.log(value); // "I'm a local variable"
}

foo();
console.log(value); // ReferenceError: value is not defined
```

Переменная `value` внутри `foo` — локальная (local variable), за пределами функции недоступна.